Low Level Design (LLD) Roadmap — 2 Hours/Day Edition (v2 — 1 task/day fixed)

Updated schedule — shifted to start Saturday, Aug 08, 2026

Goal

University ke SE/SRE/SCD/SQE/SPM/SRENG material ko use kertay hoay, hands-on practice say LLD seekhna aur ek interview-ready, GitHub-portfolio-worthy Task Management System capstone banana.

Key Decisions (locked in)

- **Primary language:** TypeScript — internship reinforcement (Angular), pairs naturally with the capstone, enough OOP (classes, interfaces, generics, access modifiers) to demonstrate SOLID/patterns clearly. Design skill is language-agnostic — transfers to Java/C# in 1-2 days if a specific interview needs it.
- **Vertical slice approach:** har milestone ke baad capstone Task Manager ko bhi update kerna hai — sirf isolated exercises nahi.
- **Daily time budget:** 2 hours — har din sirf EK system/task.
- **Total:** 46 working days (~7.5 calendar weeks, Sundays off), starting Saturday, Aug 08, 2026 → ending Wednesday, Sep 30, 2026.
- Har din + har milestone (week) ke saath ek Deadline likha hai — 6-days-on/Sunday-off cycle (agar koi din miss ho to us se agle available din 2 tasks ho jayenge — adjust as needed).
- Har din ke end par: mini deliverable + git commit — koi bhi din "sirf reading" nahi hai.
- Bonus section end mein: HLD interview questions list (ashishps1 repo se) — reference ke liye, daily schedule ka part nahi.

Daily Time Split (120 min)

Block	Time
Study (slides/book)	20-25 min
Worked Example / Practice / Challenge	80-90 min
Notes + Revision Checklist	10 min
Git Commit	5 min

Folder Reference (your directory)

- Requirements/SRS → 1- SE/Slides, 2- SRE/Books, 2- SRE/Slides
- UML → 4- SQE/Books, 4- SQE/Slides
- SOLID / Design Principles → 3- SCD/Labs/SOLID Principles.pdf, 3- SCD/Slides/Desirable Characteristics of Design.pdf
- Design Patterns → 3- SCD/Slides/Design Patterns.pdf

- Refactoring → 3- SCD/Slides/Refactoring.pdf, 3- SCD/Slides/Michael Feathers concepts.pdf, 3- SCD/Slides/Defensive Programming.pdf
- Testing → 1- SE/Slides/Lec9-ComponentTesting.pdf, Lec10-Integration&SystemTesting.pdf;, Lec11-AutomatedTesting.pdf, 3- SCD/Slides/Software Testing.pdf

WEEK 1 — Requirements → Domain Model (Days 1-8)

Milestone Deadline: Saturday, Aug 08, 2026 → Monday, Aug 17, 2026

No UML, no code. Sirf thinking clear kerni hai: problem → requirements → use cases → domain model.

Day 1 — Worked Example: Library Management

Deadline: *Saturday, Aug 08, 2026*

Read: 1- SE/Slides/Lec4-RequirementEngineering.pdf, Lec5-RequirementModeling.pdf

- Functional + Non-functional requirements likho
- Use cases identify karo
- Domain model (entities + relationships) banao

Notes/Revision: FR vs NFR ka farak apne alfaaz mein likho

`Commit: day-01-library-requirements`

Day 2 — Worked Example: ATM

Deadline: *Monday, Aug 10, 2026*

Read: 1- SE/Slides/Lec9_SRS.pdf

- Requirements + Use Cases + Domain Model (ATM ke liye)

`Commit: day-02-atm-requirements`

Day 3 — Worked Example: Food Delivery App

Deadline: *Tuesday, Aug 11, 2026*

- Requirements + Use Cases + Domain Model

Notes/Revision: teenon worked examples ka domain model compare karo — pattern nazar aa raha hai?

`Commit: day-03-food-delivery-requirements`

Day 4 — Practice: Gym Management System

Deadline: *Wednesday, Aug 12, 2026*

- Requirements
- Use Cases
- Domain Model

Do yourself, no code.

`Commit: day-04-gym-management-practice`

Day 5 — Practice: Hospital Management System

Deadline: *Thursday, Aug 13, 2026*

- Requirements + Use Cases + Domain Model

`Commit: day-05-hospital-practice`

Day 6 — Practice: Hostel Management System

Deadline: *Friday, Aug 14, 2026*

- Requirements + Use Cases + Domain Model

`Commit: day-06-hostel-practice`

Day 7 — Challenge: Netflix / Uber / LinkedIn

Deadline: *Saturday, Aug 15, 2026*

- Netflix: Actors, Features, Entities
- Uber: Actors, Features, Entities
- LinkedIn: Actors, Features, Entities

Sirf identify karo — full requirements nahi, time-boxed. Teenon light hain isliye same din theek hai.

`Commit: day-07-challenge-actors-entities`

Day 8 — Deliverable: Task Manager (SRS + Requirements)

Deadline: *Monday, Aug 17, 2026*

- SRS document
- Functional + Non-functional requirements
- User Stories
- Use Cases
- Domain Model

Capstone Phase 1 begins.

Notes/Revision: Explain "Domain Model vs Class Diagram ka farak" bina notes dekhay

`Commit: day-08-taskmanager-requirements-srs`

WEEK 2 — UML Thinking (Days 9-16)

Milestone Deadline: Tuesday, Aug 18, 2026 → Wednesday, Aug 26, 2026

Ab requirements ko diagrams mein convert kerna hai. Still no code.

Day 9 — Worked Example: Library — Class + Sequence Diagram

Deadline: *Tuesday, Aug 18, 2026*

Read: 4- SQE/Slides/Week 2.pdf, Week 3.pdf (Class Diagram basics)

- Class Diagram
- Sequence Diagram (1 flow, e.g. "issue book")

`Commit: day-09-library-uml`

Day 10 — Worked Example: ATM — Class + Sequence Diagram

Deadline: *Wednesday, Aug 19, 2026*

- Class Diagram
- Sequence Diagram ("withdraw cash" flow)

`Commit: day-10-atm-uml`

Day 11 — Worked Example: Hospital — Class + Sequence Diagram

Deadline: *Thursday, Aug 20, 2026*

- Class Diagram
- Sequence Diagram ("book appointment" flow)

`Commit: day-11-hospital-uml`

Day 12 — Practice: Bank System

Deadline: *Friday, Aug 21, 2026*

- Class Diagram
- Sequence Diagram

`Commit: day-12-bank-uml-practice`

Day 13 — Practice: School System

Deadline: *Saturday, Aug 22, 2026*

- Class Diagram

`Commit: day-13-school-uml-practice`

Day 14 — Practice: Chat App

Deadline: *Monday, Aug 24, 2026*

- Class Diagram

`Commit: day-14-chatapp-uml-practice`

Day 15 — Challenge: WhatsApp / Instagram / Spotify (pick 1, sirf UML)

Deadline: *Tuesday, Aug 25, 2026*

- Class Diagram of the chosen system

`Commit: day-15-challenge-uml`

Day 16 — Deliverable: Task Manager (Full UML Package)

Deadline: *Wednesday, Aug 26, 2026*

- Class Diagram
- Sequence Diagram (kam se kam 2 flows: "create task", "assign task")
- Package Diagram

Capstone Phase 2.

Notes/Revision: Class vs Sequence vs Package diagram — kis situation mein kaunsa use hota hai, explain bina notes

`Commit: day-16-taskmanager-full-uml`

WEEK 3 — SOLID Principles (Days 17-24)

Milestone Deadline: Thursday, Aug 27, 2026 → Friday, Sep 04, 2026

Ab coding start. Language: TypeScript.

Day 17 — Worked Example: Coffee Machine (Refactor without → with SOLID)

Deadline: *Thursday, Aug 27, 2026*

Read: 3- SCD/Labs/SOLID Principles.pdf

- "Bad" version code likho (violations ke saath)
- SOLID apply kerke refactor karo

`Commit: day-17-coffee-machine-solid`

Day 18 — Worked Example: Notification Service (Email/SMS/Push)

Deadline: *Friday, Aug 28, 2026*

- SRP + OCP focus — strategy-style structure

`Commit: day-18-notification-service-solid`

Day 19 — Worked Example: Payment Gateway (JazzCash/Stripe/PayPal)

Deadline: *Saturday, Aug 29, 2026*

- Interface-driven design, DIP focus

`Commit: day-19-payment-gateway-solid`

Day 20 — Practice: Logger

Deadline: *Monday, Aug 31, 2026*

- Code required, apply SOLID

`Commit: day-20-logger-solid-practice`

Day 21 — Practice: Authentication System

Deadline: *Tuesday, Sep 01, 2026*

- Full SOLID implementation

`Commit: day-21-auth-solid-practice`

Day 22 — Practice: File Storage System

Deadline: *Wednesday, Sep 02, 2026*

- Interface + 2 concrete implementations (local + cloud), SOLID applied

`Commit: day-22-filestorage-solid-practice`

Day 23 — Challenge: Parking Lot (or Shopping Cart — pick one, time-boxed)

Deadline: *Thursday, Sep 03, 2026*

- Full code, SOLID applied

`Commit: day-23-challenge-solid`

Day 24 — Deliverable: Apply SOLID to Task Manager

Deadline: *Friday, Sep 04, 2026*

- Task Manager ka initial code likho (TypeScript), SOLID principles ke sath

Capstone Phase 3.

Notes/Revision: SRP, OCP, LSP, ISP, DIP — har aik ko Task Manager se ek concrete example ke sath explain karo, bina notes

`Commit: day-24-taskmanager-solid`

WEEK 4 — Design Patterns (Days 25-31)

Milestone Deadline: Saturday, Sep 05, 2026 → Saturday, Sep 12, 2026

Read: 3- SCD/Slides/Design Patterns.pdf (Factory, Strategy, Observer, Builder, Adapter, Repository)

Day 25 — Worked Example: Notification Strategy Pattern

Deadline: *Saturday, Sep 05, 2026*

- Strategy pattern implement karo (Day 18 ka Notification Service extend karo)

`Commit: day-25-notification-strategy`

Day 26 — Worked Example: Payment Factory Pattern

Deadline: *Monday, Sep 07, 2026*

- Factory pattern (Day 19 ka Payment Gateway extend karo)

`Commit: day-26-payment-factory`

Day 27 — Worked Example: Document Builder Pattern

Deadline: *Tuesday, Sep 08, 2026*

- Builder pattern

`Commit: day-27-document-builder`

Day 28 — Practice: Game Characters OR Shipping (pick 1)

Deadline: *Wednesday, Sep 09, 2026*

- Observer or Factory pattern practice

`Commit: day-28-practice-pattern`

Day 29 — Challenge: Hotel Booking OR Ride Sharing (pick 1)

Deadline: *Thursday, Sep 10, 2026*

- Design discussion + partial code (Repository + Strategy/Factory mix)

`Commit: day-29-challenge-pattern`

Day 30 — Deliverable: Task Manager — Patterns Part 1

Deadline: *Friday, Sep 11, 2026*

- Repository pattern integrate karo (data access ke liye)
- Factory pattern integrate karo (task creation ke liye)

Capstone Phase 4a.

`Commit: day-30-taskmanager-patterns-1`

Day 31 — Deliverable: Task Manager — Patterns Part 2

Deadline: *Saturday, Sep 12, 2026*

- Strategy pattern (e.g. sorting/filtering tasks)
- Observer pattern (e.g. task status change notifications)

Capstone Phase 4b.

Notes/Revision: har pattern ke liye "yeh kis problem ko solve karta hai" — bina notes explain karo

`Commit: day-31-taskmanager-patterns-2`

WEEK 5 — Refactoring + Testing (Days 32-43)

Milestone Deadline: Monday, Sep 14, 2026 → Saturday, Sep 26, 2026

Day 32 — Worked Example: God Class Refactor

Deadline: *Monday, Sep 14, 2026*

Read: 3- SCD/Slides/Refactoring.pdf

- God Class identify → refactor karo

`Commit: day-32-god-class-refactor`

Day 33 — Worked Example: Long Method Refactor

Deadline: *Tuesday, Sep 15, 2026*

Read: 3- SCD/Slides/Michael Feathers concepts.pdf

- Long Method refactor

`Commit: day-33-long-method-refactor`

Day 34 — Worked Example: Duplicate Code Refactor

Deadline: *Wednesday, Sep 16, 2026*

- Duplicate Code refactor

`Commit: day-34-duplicate-code-refactor`

Day 35 — Practice: Student System OR Payroll (pick 1)

Deadline: *Thursday, Sep 17, 2026*

- Refactor exercise

`Commit: day-35-refactor-practice`

Day 36 — Challenge: Legacy Hospital / CRM / POS (pick 1)

Deadline: *Friday, Sep 18, 2026*

- Legacy-style messy code ko refactor karo

`Commit: day-36-challenge-legacy-refactor`

Day 37 — Deliverable: Task Manager Refactor

Deadline: *Saturday, Sep 19, 2026*

- Puray Task Manager codebase ka refactoring pass

Capstone Phase 5.

`Commit: day-37-taskmanager-refactor`

Day 38 — Worked Example: Calculator — Unit Tests

Deadline: *Monday, Sep 21, 2026*

Read: 1- SE/Slides/Lec9-ComponentTesting.pdf

- Unit tests likho

`Commit: day-38-calculator-tests`

Day 39 — Worked Example: Login Module — Unit Tests

Deadline: *Tuesday, Sep 22, 2026*

- Unit tests likho

`Commit: day-39-login-module-tests`

Day 40 — Worked Example: Shopping Cart — Unit + Integration Tests

Deadline: *Wednesday, Sep 23, 2026*

Read: 1- SE/Slides/Lec10-Integration&SystemTesting.pdf;

- Unit + integration tests

`Commit: day-40-shopping-cart-tests`

Day 41 — Practice: ATM OR Bank OR Todo (pick 1)

Deadline: *Thursday, Sep 24, 2026*

- Tests likho

`Commit: day-41-practice-tests`

Day 42 — Challenge: Chess OR Booking OR Inventory (pick 1)

Deadline: *Friday, Sep 25, 2026*

- Tests likho

`Commit: day-42-challenge-tests`

Day 43 — Deliverable: Task Manager — Unit + Integration Tests

Deadline: *Saturday, Sep 26, 2026*

- Unit tests (core logic)
- Integration tests (repository + service layer)

Capstone Phase 6.

Notes/Revision: Unit vs Integration test ka farak, apne code se example dekar explain karo

`Commit: day-43-taskmanager-tests`

WEEK 6 — Final Capstone Assembly (Days 44-46)

Milestone Deadline: Monday, Sep 28, 2026 → Wednesday, Sep 30, 2026

Day 44 — Capstone Review: Requirements → UML

Deadline: *Monday, Sep 28, 2026*

- Phase 1-2 review: SRS, Use Cases, Domain Model, Class/Sequence/Package diagrams — sab ek README mein consolidate karo

`Commit: day-44-capstone-docs-consolidation`

Day 45 — Capstone Review: Implementation → Patterns

Deadline: *Tuesday, Sep 29, 2026*

- Phase 3-4 review: code clean karo, SOLID + patterns ka final check
- README mein architecture diagram/explanation add karo

`Commit: day-45-capstone-code-review`

Day 46 — Capstone Finalize: Testing + Portfolio Polish

Deadline: *Wednesday, Sep 30, 2026*

- Phase 5-6 review: refactoring + tests final check, test coverage note karo
- GitHub README polish (badges, setup instructions, architecture overview)

`Commit: day-46-capstone-final`

Capstone complete ho jane ke baad tumhare paas hoga: Requirements ■ → SRS ■ → Use Cases ■ → Domain Model ■ → Class Diagram ■ → Sequence Diagram ■ → Full TypeScript Implementation ■ → SOLID ■ → 4 Design Patterns ■ → Refactored Code ■ → Unit + Integration Tests ■

Reference Section

Coverage Summary

System	Requirements	UML	Code	Tests
Library	■	■	—	—
ATM	■	■	—	■ (practice)
Food Delivery	■	—	—	—
Gym / Hospital / Hostel	■	Hospital only	—	—
Bank / School / Chat App	—	■	—	■ Bank
Coffee Machine / Notification / Payment	—	—	■ SOLID	—
Logger / Auth / File Storage	—	—	■ SOLID	—
Parking Lot / Shopping Cart	—	—	■ SOLID	—
Notification Strategy / Payment Factory / Document Builder	—	—	■ Patterns	—
Ride Sharing / Hotel Booking	—	Discussion	■ Patterns	—
Legacy Hospital/CRM/POS	Existing	Existing	■ Refactor	—
Calculator / Login / Shopping Cart	—	—	—	■
Task Manager (Capstone)	■	■	■	■

Language Note

Roadmap TypeScript mein hai. Agar kabhi kisi specific company/interview ke liye Java ya C# chahiye ho, wahi design (classes, SOLID, patterns) usi din 1-2 ghante mein translate ho jayega — concepts same rahenge, sirf syntax badlega.

SOLID Quick Checklist

- SRP — class ki ek hi wajah honi chahiye badalne ki
- OCP — extension ke liye open, modification ke liye closed
- LSP — subclass, parent class ki jagah use ho sakni chahiye bina behavior break kiye
- ISP — chhoti, specific interfaces — client ko unused methods implement na kerne padein
- DIP — high-level modules, low-level modules par directly depend na karein — abstraction par depend karein

Design Pattern Cheat Sheet

Pattern	Problem it solves
Factory	Object creation logic ko caller se hide kerna
Strategy	Runtime par algorithm/behavior switch karna
Observer	Ek object change ho to dependents ko notify kerna
Builder	Complex object ko step-by-step construct kerna
Adapter	Incompatible interfaces ko compatible banana
Repository	Data access logic ko business logic se separate kerna

Common Mistakes to Avoid

- Design patterns ko "force fit" karna jahan simple code kaafi ho
- UML diagrams ko implementation se pehle detail mein perfect kerne ki koshish (iterate karo, perfect mat socho)
- SOLID ko rules ki tarah follow kerna bina "kyun" samjhay
- Tests ko end mein "extra kaam" samajhna instead of design ka part

Interview Prep Questions (self-test after each week)

- Week 1-2: "Requirements se Class Diagram tak kaise pohnchoge? Domain model kyun zaroori hai?"
- Week 3: "SRP aur SOC (Separation of Concerns) mein farak?"
- Week 4: "Factory vs Builder — kab kaunsa use karoge?"
- Week 5-6: "Refactoring aur re-engineering mein farak? Legacy code ko safely refactor kaise karoge (Michael Feathers approach)?"

Bonus: System Design (HLD) Interview Questions — for later

Yeh LLD roadmap se alag hai (isliye daily schedule mein nahi daala) — unified roadmap mein HLD phase hackathon ke baad already scheduled hai. Yahan sirf reference ke liye rakh raha hoon taake list haath mein rahe jab wo phase aaye.

Source: ashishps1/awesome-system-design-resources — core concepts, networking, API/database/caching fundamentals, distributed systems, aur tradeoffs bhi is repo mein hain (worth a full read during the HLD phase), lekin neeche sirf interview problems list hai jo directly practice-worthy hai.

Easy

- Design URL Shortener (TinyURL)
- Design Autocomplete for Search Engines
- Design Load Balancer
- Design Content Delivery Network (CDN)
- Design Parking Garage
- Design Vending Machine
- Design Distributed Key-Value Store
- Design Distributed Cache
- Design Authentication System
- Design Unified Payments Interface (UPI)

Medium

- Design WhatsApp
- Design Spotify
- Design Instagram
- Design Notification Service
- Design Distributed Job Scheduler
- Design Tinder
- Design Facebook
- Design Twitter
- Design Reddit
- Design Netflix
- Design YouTube
- Design Google Search
- Design E-commerce Store (Amazon-style)
- Design TikTok
- Design Shopify
- Design Airbnb
- Design Rate Limiter
- Design Distributed Message Queue (Kafka-style)
- Design Flight Booking System
- Design Online Code Editor
- Design an Analytics Platform (Metrics & Logging)
- Design Payment System

- Design a Digital Wallet

Hard

- Design Location Based Service (Yelp-style)
- Design Uber
- Design Food Delivery App (Doordash-style)
- Design Google Docs
- Design Google Maps
- Design Zoom
- Design File Sharing System (Dropbox-style)
- Design Ticket Booking System (BookMyShow-style)
- Design Distributed Web Crawler
- Design Code Deployment System
- Design Distributed Cloud Storage (S3-style)
- Design Distributed Locking Service

Note: Parking Garage aur Vending Machine already tumhare LLD scope se overlap karte hain (Parking Lot Day 23 challenge mein hai) — jab HLD phase aaye to unko "scale + infra" angle se dobara dekhna, LLD wale class design se zyada.